Developer notes

Background for building The Village booking system. Business decisions, business rules and build details live in the requirements tab; this tab is how to build them.

Start here (for Claude Code)

This doc is the brief. In the first session, do these in order and don't write app code yet:

1. **Read both tabs in full:** the requirements tab (what to build) and these Developer notes (how to build it). Also open the wireframe screenshots and mockup screenshots in the project folder.
2. **List what's still open** and show it to the owners: Open questions, rows marked **Decide**, unticked rows in Mockup vs requirements, and anything in the doc that conflicts. Ask before guessing. Use clearly marked placeholders (prices, opening hours, wording, pictures) until the owners supply them.
3. **Set up the project** as in step 1 of How we build: local folder, private GitHub repo, `CLAUDE.md` with this doc's rules, and a first commit and push. Give the owners their checklist of accounts and purchases.
4. **Start the design phase:** wireframes for every customer and staff screen, shared on a preview link for the owners to review.
5. **Stop for sign-off** before any app code, database tables or paid services are set up.

Where the two tabs disagree, the requirements tab wins on what to build, and these notes win on how to build it. Mockup vs requirements follows its own rule: requirements win unless a row is ticked Mockup.

Recommendation

Build The Village from scratch as its own app, designed for its sessions, packs and roles, with its own database and a phone-friendly customer booking site. No code or data is copied from any other studio's system. Running costs are about $76–87 a month; hours are re-estimated once the designs are signed off.

- **Why from scratch:** a clean design around The Village's rules (credit reservation, open play, private hire, workshops, two staff roles) instead of adapting another studio's app.
- **What it isn't:** a full studio platform. No online payments, waitlists or app-store app at launch.

- **The alternative:** studio software such as Momence, bsport, TeamUp or Glofox, which include online payments, waitlists and a customer app for a monthly fee (prices not checked). Worth it if online payment or a native app is needed within the year.

How we build

Nothing is coded until the owners have seen and approved the wireframes and visuals. Claude Code does the setup and the build, and guides the owners through every step that needs them.

1. **Set up (Claude Code).** Creates a local project folder on the laptop (for example ~/Projects/the-village) and a private GitHub repository, linked to each other. Sets up branch protection on `main`, pull-request checks (tests, lint, type checks), Dependabot, and a `CLAUDE.md` that holds this doc's rules. Connects two Vercel projects and two Supabase projects (staging and production). Gives the owners a checklist of what only they can do: buy Vercel Pro and the domain, open the Supabase, Resend, Meta and Zalo accounts, and invite the developer.
2. **Design (Claude Code, owners review).** Wireframes for every customer and staff screen, then the visual style (pastel colours, type, the session pictures), then a clickable prototype on a Vercel preview link the owners can open on their phones.
3. **Finalise (owners sign off).** Owners approve the designs and this doc. The data model, the list of screens and the hour estimates are then fixed; later changes go into a change list and are agreed before they're built.
4. **Build (Claude Code).** One phase at a time, each as a pull request with a preview link. The owners check each preview and approve it before it's merged and goes live.
5. **Test and pilot.** Automated tests on booking, credits and access, then a pilot with 5–10 customers on staging data, then launch.

How Claude Code guides us:

- Explains each step in plain words before doing it, and what the owners need to check.
- Asks before anything that costs money, creates an account, sends messages to customers or changes production.
- Never merges to `main` or changes the production database without an owner's approval.
- Keeps a short progress note in the repo: what's done, what's next, what's waiting on the owners.

Keeping the work safe: the laptop is not a safe place to keep it, so GitHub is the master copy.

- Claude Code commits and pushes to GitHub at the end of every working session and after each finished step, never leaving work only on the laptop.
- Everything goes in the repo: code, database migrations and seed data, the wireframes and design files, `CLAUDE.md`, and the progress note.
- Secrets never go in the repo. Keys live in Vercel and Supabase environment settings; a `.env.example` lists their names without values, and `.env` files are in `.gitignore`.
- Work in progress goes on a branch and is pushed too, so nothing is lost if the laptop is lost or broken. A new laptop only needs to clone the repo.

Decide: the date the designs must be signed off to keep the 14 November opening.

Who can do what

Action	Manager	Instructor	Customer
See the timetable	Yes	Yes	Yes, with spaces left
Book someone into a class	Anyone	Anyone	Themselves only
Cancel a booking	Anyone, any time	Anyone, any time	Their own, before the cutoff
See customer list and details	Yes	Own sessions only, or all if a manager allows	Their own only
Sell packages, one-offs, trials	Yes	No	No
Adjust credits manually	Yes	No	No
Create and edit classes and the schedule	Yes	Own classes only	No
Add private hire to the calendar	Yes	No	Message us only
Catalog (class types, prices, packages)	Yes	No	No

Action	Manager	Instructor	Customer
Reports and revenue	Yes	No	No
Manage staff accounts	Yes	No	No
Check in and mark attendance, no-shows, late cancellations	Yes	Yes	No
Record payments	Yes	No	No

Agreed with the owners. A manager can also be an instructor.

Session types

Type	Example	Who creates it	Capacity	How customers join	Paid with
Group class	Pilates Flow, 9:00–10:00, 7/10 booked	Staff, often as a weekly recurring class	Up to 10 parents, each with up to 2 children	Book a space on the customer site	Class pack credit, one-off or trial
Open play	Open Play, 10:30–12:00, 5/10 booked	Staff	Up to 10 children; up to 2 parents per child free	Book a space, like a class	One open play credit or 150,000 VND per child
Private hire	Studio hire, 13:00–15:00	Staff, after agreeing it by chat	Whole room, one customer	Message us on WhatsApp; staff add it in any slot without a class	Agreed in person
Workshop or event (M7)	Child sleep talk, 10:00–12:00, 12/20 booked	Managers, one-off	Set per session	Book a ticket, like a drop-in	Its own price, paid on the day or by transfer; never pack credits

All four share one calendar and one room, so two sessions can never overlap. The database enforces this (see Data model).

Core rules the build must get right

These are the parts most likely to go wrong; each has automated tests (P3).

1. **Credit reservation.** A booking reserves a credit straight away, so a customer with 1 credit can't book several sessions. The credit is used once the session ends.
2. **Automatic attendance.** Attendance and credit deduction run on a schedule, not when staff open the app (see Scheduled jobs).
3. **Safe booking.** The last space and duplicate bookings are handled in the database, so two people tapping Book at once can't both get the last place.
4. **Booking rules.** Use the pack that expires first among packs valid on the session date; one intro pack and one free trial per customer; one parent with up to 2 children per class booking, and one credit per child for open play; guests limited to one unpaid booking, logged-in customers not; guest bookings matched to existing customers by phone or email. Full list under Build details in the requirements tab.
5. **Studio settings, not hard-coding.** Studio name, contact details, booking rules and messaging links live in settings.
6. **Vietnam time.** Store and schedule everything in Asia/Ho_Chi_Minh (UTC+7).
7. **Calendar.** Day, week and month views, drag to move, and no two sessions overlapping in the one room.

Effort by phase

These hours assumed reusing an existing app. Building from scratch, with a design phase first, will take longer; Claude Code re-estimates each phase once the designs are signed off.

Phase	What	Hours
1. Village setup	New database and website, pastel branding, paid plans, domain, email	4–6
2. Staff roles	Manager and Instructor roles, staff account page	3–5
3. Booking rules	Credit reservation, class types, class and open play packs, private hire as a session type, free trials, cancellation cutoff, automatic attendance	7–11

Phase	What	Hours
4. Customer site	Email code login, timetable, booking, my bookings, my packages, private hire WhatsApp button, contact details, self sign-up with a pay-on-the-day first booking, home-screen icon	18–25
5. Languages and calendars	English/Vietnamese switch, calendar links for customers and instructors	7–11
6. Public website	One-page site with a Book now button	3–6
7. Safety, operations and reports	Optional child name and birth month (autofilled), session details, check-in list, cancelling a session for everyone, instructor swap and notes, closures, pausing packs, pack types and promotions, studio settings, welcome emails, staff sign-in, data deletion, money and instructor reports, privacy and terms pages, messaging buttons	22–36
8. Test and pilot	Automated tests, pilot with 5–10 customers, fixes	6–10
Total		70–110 **(**76–123 **with later additions)**

Split by release: about 53–85 hours for opening day (14 November, including WhatsApp/Zalo login and guest booking) and 23–38 hours for the release two weeks later. That includes items added after this table was drawn up: reminder emails and session pictures (opening day), and multi-date bookings, and workshops (second release). Rough per-requirement estimates: messaging buttons and small staff tools under 1 h each; self sign-up with pay-on-the-day booking 5–8 h; English/Vietnamese 4–6 h; calendar links 3–5 h; website 3–6 h.

Running costs

About $76–87 a month. Point-in-time recovery is left out for now (decided 20 October); adding it later costs about $105 a month more ($100 for 7 days of recovery plus a $5 compute upgrade).

Item	Monthly cost (USD)	Notes
Vercel Pro	$20	Includes 1 developer seat and $20 of usage credit. Extra developer seats $20 each; viewer seats free. Both Vercel projects sit in one team at no extra cost
Supabase Pro, production project	$25	Includes $10 of compute credit (covers the Micro size), daily backups kept 7 days, never paused
Supabase staging project	$10	A second project on Micro compute
Resend Pro (email)	$20	The free tier is 3,000 a month but only 100 a day; reminders, confirmations and login codes on a busy day can pass 100, and login emails would stop. Pro gives 50,000 a month with no daily limit
Domain name	About $1–2	About $10–20 a year (not checked)
WhatsApp, Zalo and optional SMS login messages	About $0–10	Pay per message once approved; a verified Zalo Official Account tier is extra (not checked)

Checked on 20 October 2026 on the Vercel, Supabase, Supabase point-in-time recovery and Resend pricing pages; USD before tax. An inbox on the domain (Google Workspace or Zoho) is separate and optional.

Domains and email setup

.vn and .com.vn domains are bought through a Vietnamese registrar such as Mat Bao, PA Vietnam or iNET; a .com from any registrar. Automated emails go through Resend (or similar) from an address on the domain, after adding its DNS records at the registrar so emails don't land in spam.

Staff app features

1. "Call to confirm" list on the dashboard: new customers and unpaid drop-ins, with a Called tick.
2. Session types on the calendar, each with its own colour.
3. Roles: instructors don't see Catalog, Reports or the Staff page.
4. Staff page for managers to add, remove and reset staff accounts.
5. Payment method and amount recorded on every sale, drop-in, workshop and private hire, by managers only.

Tech stack

A new codebase on Next.js, Supabase and Vercel, written from scratch. Claude Code records these choices in the repo's `CLAUDE.md` at setup.

Layer	Choice	Notes
Framework	Next.js (App Router) with TypeScript	Latest stable version at setup, pinned in the repo
Database and auth	Supabase (Postgres, Auth, Storage, Realtime)	New projects for staging and production; nothing shared with other systems
Hosting	Vercel Pro	See Hosting on Vercel
Email	Resend, through Supabase Auth's custom SMTP too	Supabase's built-in email is rate-limited; never use it in production
Email templates	React Email, one template per language	English and Vietnamese, chosen by the customer's language

Layer	Choice	Notes
Translations	next-intl	Customer site and emails only; staff app is English
Session pictures (C28)	Supabase Storage, public bucket, resized on upload	Managers upload; customers read
Calendar feeds (C11, S8)	An `.ics` route per user, behind a secret token	Works in Google, Apple and Outlook
Bot protection	Cloudflare Turnstile (or Vercel BotID)	On guest booking and login forms
Error tracking	Sentry (free tier)	Vercel keeps logs only briefly
Uptime	A free uptime monitor on the booking site and a health route	Alerts to the developer
Tests (P3)	Vitest for rules and database functions, Playwright for the booking flow	Run in CI on every pull request

Two Vercel projects (decided 20 October):

Vercel project	Hostname	Holds
Website	`thevillagesaigon.com`	The one-page website (W1)
Booking app	`book.thevillagesaigon.com` and `staff.thevillagesaigon.com`	Customer booking site and staff app, one Next.js app routed by hostname, sharing one database

Home-screen icon (C9): on iPhone, a site added to the home screen has its own storage, separate from Safari, so the parent must log in again inside it. After the first booking, prompt "Add to home screen" before the parent logs in on that phone, or tell them they'll log in once more.

Hosting on Vercel

Vercel Pro hosts all three hostnames; the database stays on Supabase, in the same region. Prices and limits below are from memory; check them on sign-up.

- **Plan and seats.** Pro is billed per team member. Only developers need seats; owners don't need one to use the app.
- **Owner of the accounts.** Vercel, Supabase, Resend, the registrar, Meta and Zalo accounts are opened in The Village's name with an owner's email, and the developer is invited. MFA on all of them.
- **Region.** Supabase in Singapore (ap-southeast-1) and Vercel functions in Singapore (`sin1`), so each request doesn't cross an ocean.
- **One bill (optional).** Supabase can be added through the Vercel Marketplace and billed by Vercel. Check that daily backups and Pro limits come with it before choosing this.
- **Spend cap.** Turn on Vercel spend management with a monthly limit and alerts.
- **Firewall.** Turn on the Vercel Firewall; use attack challenge mode if bots hit the guest booking form.

Environments:

Environment	Database	Deploys from	Used for
Local	Supabase CLI, local Docker	Developer machine	Building and tests
Preview	Separate staging Supabase project	Every pull request	Owners review changes; pilot rehearsal
Production	Production Supabase project	`main` only	Live studio

- Preview deployments must never hold production database keys. Set environment variables per environment, and mark keys as Sensitive.
- Database changes go through Supabase migration files in the repo; CI runs migrations and tests on staging before merging.
- Seed data (class types, packs, studio settings) lives in a seed file, so staging can be rebuilt.

Data model

Credits are an append-only ledger, and the database itself enforces capacity, overlap and one booking per person. Draw the full schema (an ERD in the repo) before writing booking code.

Main tables:

Table	Holds	Key rules
`staff_roles`	Staff user id, role (manager, instructor), instructor access (own sessions only or all customers)	Role is read from here or `app_metadata`, never `user_metadata`
`studio_settings`	Studio name, contact details, messaging links, cancellation cutoff, booking window, latest booking time, reminder time	One row; read by the app and jobs, never hard-coded. Editable by managers with M9
`session_types`	Class, open play, private hire, workshop; colour, picture	
`pack_types`	Size, price, validity days, intro or promotion flags, start and end dates	Linked to the session types it covers
`customer_packs`	A pack sold to a customer, price paid, valid from and to, paused days	Price copied at sale; later price changes don't touch it
`credit_ledger`	One row per credit movement: reserve, use, return, forfeit, bonus, restore	Never updated or deleted; balance = sum of rows. Each row has who, when and a reason
`bookings`	Customer, session, pack used, status, source (online, staff, guest), unpaid flag, called tick	Unique on (session, customer) while active
`recurring_series`	Customer, class, weekday, status (active, paused, ended)	Generates bookings 4 weeks ahead
`payments`	Amount, method (cash, transfer, QR), what it paid for, recorded by	Managers only; kept when a customer is anonymised
`customer_children`	Child name, birth month and year	Optional; deleted on request
`closures`, `pack_pauses`	Dates closed; pauses with reason	M1, M2

Table	Holds	Key rules
`login_challenges`	WhatsApp and Zalo login codes, phone, browser, expiry, attempts	Short expiry; single use
`calendar_tokens`	Secret token per user for the `.ics` feed	Revocable
`notifications`	Every email or message sent, type, status	Stops the same reminder going twice
`audit_log`	Staff actions on credits, payments, customers, roles	Who, what, before and after

Rules the database enforces (Postgres, not just app code):

1. **Booking is one database function.** It locks the session row, checks capacity, picks the pack that expires first, writes the booking and the reserve ledger row in one transaction. The app calls only this function.
2. **No overlap.** An exclusion constraint on each session's time range (`btree_gist`), since there is one room.
3. **Customers are unique by phone and email.** Phones stored in international form (+84…), emails in lowercase, unique indexes on both. If a guest's phone matches one customer and email matches another, book under the email match and flag it for a manager.
4. **Time.** All times are `timestamptz`. "Valid on the session date" compares dates in Asia/Ho_Chi_Minh.
5. **Money.** VND stored as whole numbers (`bigint`), no decimals.
6. **Row-level security on every table** (see Security and access).
7. **Deletion (M10).** Anonymise the customer row (name, phone, email cleared) and delete children; bookings, ledger and payments keep the customer id. Design the foreign keys for this now.

Backups (decided 20 October):

- **Daily backups** on Supabase Pro, kept 7 days. Point-in-time recovery is not used for now (decided 20 October); it can be added later if needed.

- **Weekly Excel backup by email.** A scheduled job exports customers, packs, credit balances, bookings and payments to an Excel file and emails it to the two managers only. The file is password-protected, with the password shared once outside email.
- One test restore to staging before opening day.

Reports (M3, M4) read from database views over `payments`, `bookings` and `credit_ledger`, not from app code.

Scheduled jobs

Jobs run in the database with Supabase pg_cron, not Vercel cron: Vercel cron schedules are in UTC and don't retry a failed run. Jobs that send email call a protected Vercel route or Supabase Edge Function.

Job	Vietnam time	Cron (UTC)	Does
Close finished sessions (S6)	Every 15 min	`*/15 * * * *`	Marks unmarked bookings attended; turns reserved credits into used
Day-before reminders (C14)	18:00 daily	`0 11 * * *`	Emails everyone booked for tomorrow, in their language
Clean up login codes	Hourly	`0 * * * *`	Deletes expired login challenges
Weekly Excel backup	Monday 07:00	`0 0 * * 1`	Exports customers, packs, credit balances, bookings and payments to a password-protected Excel file and emails it to the two managers

Every job must:

- be safe to run twice (check `notifications` and booking status before acting)
- write a row per run with its result, and alert the developer by Sentry on failure
- work only on rows it locks, so two runs can't overlap

Security and access

The "Who can do what" table is enforced by row-level security and database functions, not by hiding pages. Hiding Catalog or Reports from instructors is only the UI layer; the API must refuse those actions too.

Row-level security by role:

Data	Customer	Instructor	Manager
Sessions, timetable	Read	Read; edit own classes	All
Own profile, children, packs, bookings	Read; book and cancel through functions	—	—
Other customers	None	Own sessions' people only, or all customers, as a manager sets per instructor; search to book (S12)	All
Credit ledger	Read own	Write attendance rows only, through functions	All, through functions
Payments, catalog, reports, staff	None	None	All
Audit log	None	None	Read

Instructor access (decided 20 October): on the Staff page, a manager sets each instructor to **Own sessions only** (the default) or **All customers**. Own sessions only means they see the customers and child details of people booked into their sessions, plus name search when booking someone in (S12). The setting is enforced by row-level security and logged in the audit log.

Login and accounts:

- **Email codes (C1):** 6 digits, 10 minutes, 5 attempts, then a new code; rate limit per email and per IP.
- **WhatsApp and Zalo login codes (C30):** a 6-digit code sent with Meta's authentication template (or Zalo ZNS) to the phone number entered, valid for 5 minutes, at most 5 tries, rate-limited per phone and per IP. A correct code verifies the phone (decided 21

Oct: phone number, then code). Verify Meta's webhook signature (`X-Hub-Signature-256`) on every message.

- **Phone is trusted only once verified.** A guest-entered phone (C15) is stored as unverified; WhatsApp or Zalo login can match only a verified phone. It becomes verified when the parent logs in by WhatsApp or Zalo, or staff confirm it on the call.
- **One-year sessions:** refresh tokens rotate; customers get "Log out everywhere"; staff can sign a customer out.
- **Staff:** individual accounts, MFA for managers, demo passwords removed (S7), staff accounts created only by managers or the developer.
- **Roles** live in `staff_roles` or `app_metadata`; never in `user_metadata`, which users can change themselves.

Forms and links:

- Guest booking and login forms have Turnstile, plus rate limits by IP, phone and email.
- The guest form never reveals whether an email or phone already has an account.
- Cancel links in emails and calendar feed links use random, single-purpose tokens; cancel links expire at the cutoff. Calendar feeds show session titles only, no names.

Secrets and code:

- New Supabase projects and new keys; no keys reused from any other project.
- The Supabase service-role key is server-only, never sent to the browser.
- Security headers and a Content Security Policy on all three hostnames; Dependabot on the repo.
- Domain: SPF, DKIM and DMARC for email; registrar lock on the domain.

Data law (B1): Vietnam's Decree 13/2023 and the 2026 Personal Data Protection Law. The owners checked: hosting in Singapore needs no filing. Still build for clear consent, extra care with children's data, requests to see or delete data (M10), and a plan to report a data breach quickly.